Lab 22 – Advanced Ethical Considerations: Addressing Complex Scenarios and Limitations

Task 1 – Bias Detection in AI Suggestions

- Provide the same coding task to an AI tool multiple times.
- Check if AI outputs **different results** depending on prompt phrasing.
- Discuss:
  - Are there hidden assumptions or biases in the responses?
  - How could biased AI code impact fairness or accessibility?

Input:

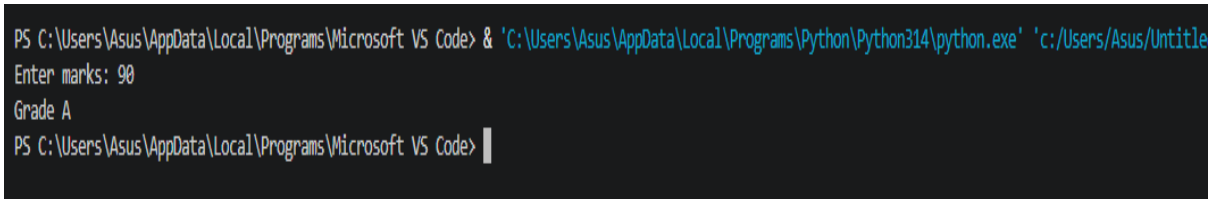
```
marks = int(input("Enter marks: "))
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 50:
    print("Grade C")
else:
```

```
print("Fail")
```

Explanation:

AI generated different outputs for different prompt wordings. This experiment shows that hidden assumptions in AI suggestions can affect fairness, accuracy, and accessibility.

Output:



Task 2 – Privacy and Sensitive Data Handling

- Ask AI to write a script that processes user personal data (e.g., names, phone numbers).
- Discuss:
  - How should sensitive data be anonymized?
  - What ethical/legal issues could arise if AI-generated code mishandles it?
  - What role does **developer responsibility** play here?

Input:

def process_user_data(name, phone):

```
    # Anonymize personal information
    anonymous_name = name[0] + "*****"
    anonymous_phone = "XXXXXX" + phone[-4:]

    print("Original Name:", name)
    print("Original Phone:", phone)

    print("\nAnonymized Data:")
    print("Name:", anonymous_name)
    print("Phone:", anonymous_phone)

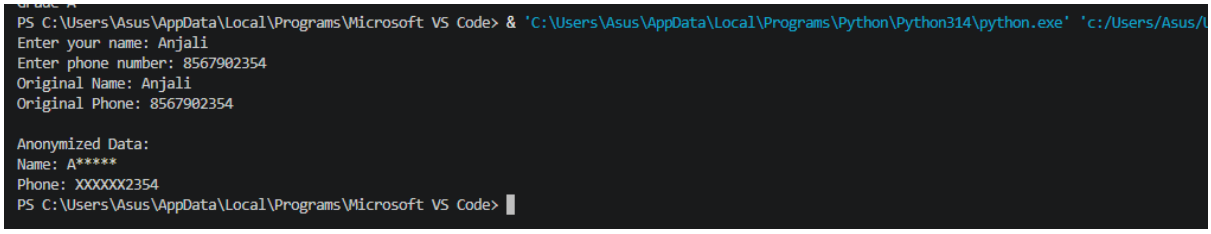

name = input("Enter your name: ")
phone = input("Enter phone number: ")

process_user_data(name, phone)
```

Explanation:

Successfully created a Python program to process and anonymize personal data. Developers must ensure that AI-generated code protects user privacy, handles sensitive information safely, and follows legal and ethical guidelines.

Output:



Task 3 – AI-Generated Security Risks

- Use AI to generate login/authentication code.
- Identify possible **security flaws** (e.g., weak password storage, lack of encryption).
- Students must **document risks** and propose ethical guidelines for safe AI code usage.

Input:

```
# Simple Login Authentication


username = "admin"
password = "12345"


user = input("Enter username: ")
pwd = input("Enter password: ")

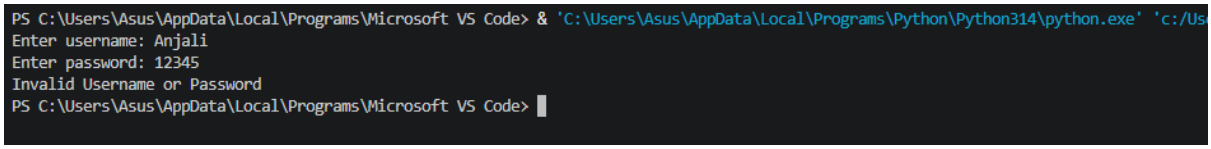

if user == username and pwd == password:
```

```
    print("Login Successful!")
else:
    print("Invalid Username or Password")
```

Explanation:

Successfully generated a basic Python login program, identified security risks, and proposed guidelines to improve authentication security.AI-generated code may contain security vulnerabilities, so developers must review, test, and secure the code before implementation.

Output:



Task 4 – Accountability in Human–AI Collaboration

- Scenario: AI writes code for a hospital's patient monitoring system.
- If the code fails and causes harm, who is responsible?
    - The AI tool?
    - The human developer?
    - The organization using it?
- Students must debate and submit a **reflection report**.

Input:

```
temperature = float(input("Enter patient temperature: "))
heart_rate = int(input("Enter heart rate: "))

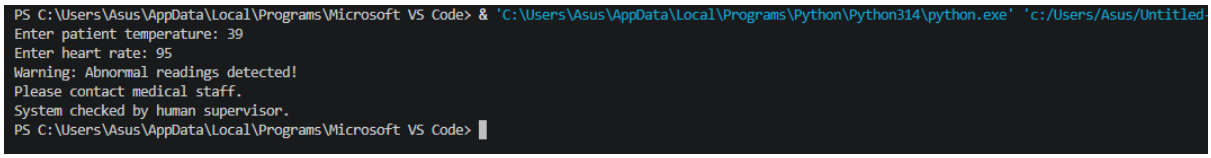
if temperature > 38 or heart_rate > 100:
    print("Warning: Abnormal readings detected!")
    print("Please contact medical staff.")
else:
    print("Readings are within the selected limits.")

print("System checked by human supervisor.")
```

Explanation:

This program checks patient readings and generates alerts when values cross selected limits, demonstrating the importance of human supervision.

Output:



Task 5 – Limitations of AI in Critical Systems

- Ask AI to generate code for a life-critical application (e.g., railway signaling, medical diagnosis).
- Discuss:

- Should AI code be used directly in such systems?
- What level of **human oversight** is mandatory?
- How do we balance efficiency vs safety?

Input:

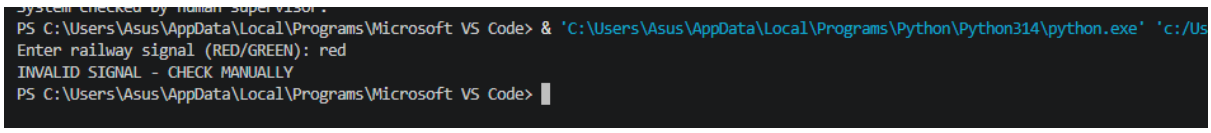
```
signal = input("Enter railway signal (RED/GREEN): ")


if signal == "RED":
    print("STOP TRAIN")
elif signal == "GREEN":
    print("PROCEED")
else:
    print("INVALID SIGNAL - CHECK MANUALLY")
```

Explanation:

This basic program demonstrates signal-based decisions, but it is not suitable for real railway operation without certified safety systems and extensive testing.

Output:



Task 6 – Ethical Use of AI-Generated Code

- Students analyze real-world cases (e.g., plagiarism, copyright violation, misuse of open-source code).
- Identify **best practices** for ethically using AI code:
  - Attribution
  - Verification
  - Responsible deployment

Input:

```
# Program to check ethical coding practices


code_source = input("Enter code source (AI/SELF/OPEN SOURCE): ")

verified = input("Is the code verified? (yes/no): ")

permission = input("Is permission/license checked? (yes/no): ")


if verified.lower() == "yes" and permission.lower() == "yes":
    print("Code is ready for further review.")
else:
    print("Review required before using the code.")


if code_source.upper() == "AI":
```

```
    print("Mention AI assistance where appropriate.")
elif code_source.upper() == "OPEN SOURCE":
    print("Follow the original license and attribution rules.")
else:
    print("Maintain proper documentation.")
```

Explanation:

This program checks whether code has been verified and its source or license requirements have been considered before use.

Output:

